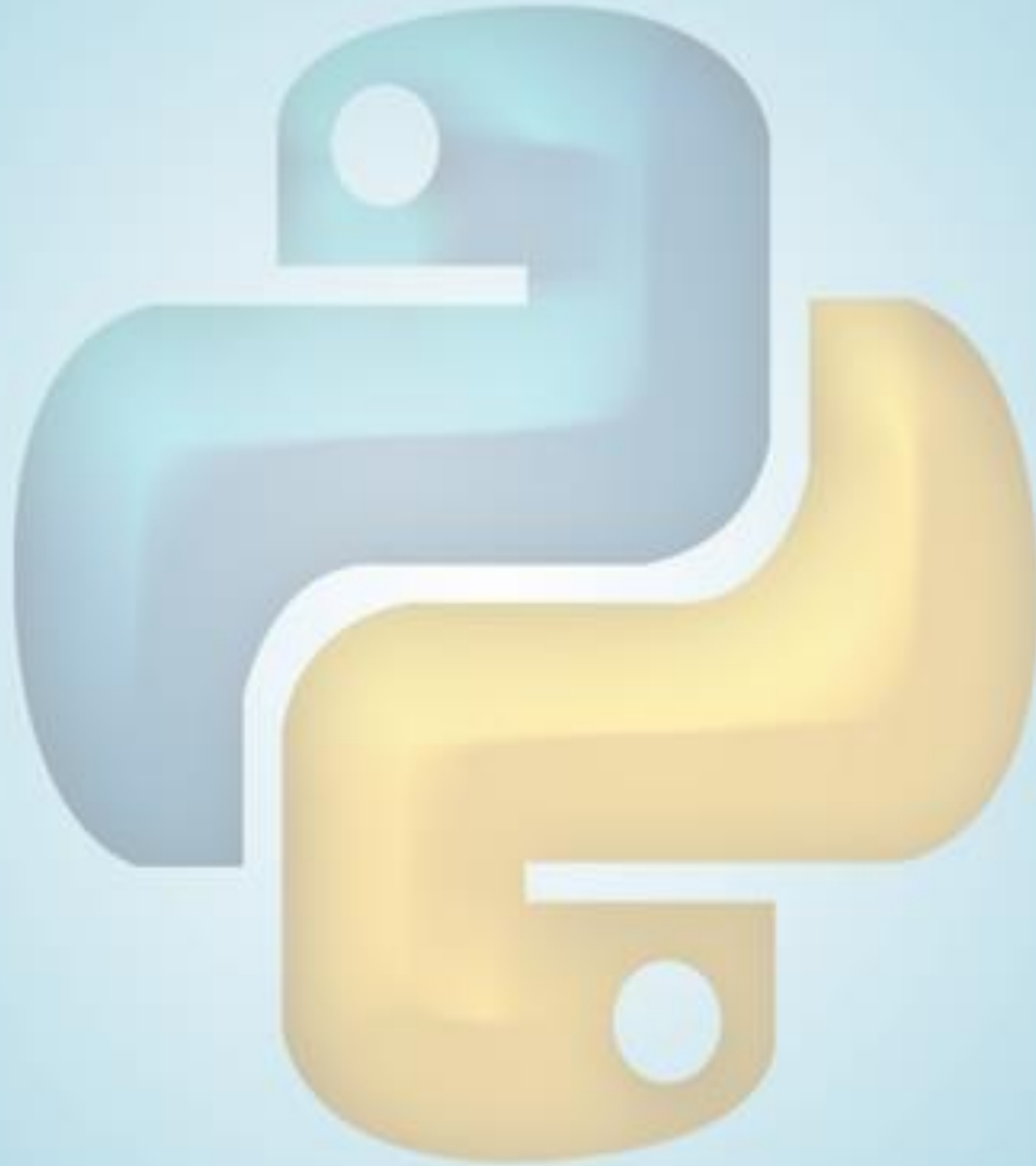


PYTHON PROGRAMMING COURSE



OUTLINE

- Getting PIP
- Installation Pygame
- Starter
- Create Background Color
- Adding Ship Object
- Refactoring
- Exercise



GETTING PIP

- De facto standard package management system
- Checking pip

```
Last login: Sat Jun 29 05:38:56 on ttys000
[thiris-air-2:~ thiri$ python -m pip --version
pip 8.1.2 from /Library/Python/2.7/site-packages/pip-8.1.2-py2.7.egg (python 2.7
)
thiris-air-2:~ thiri$
```

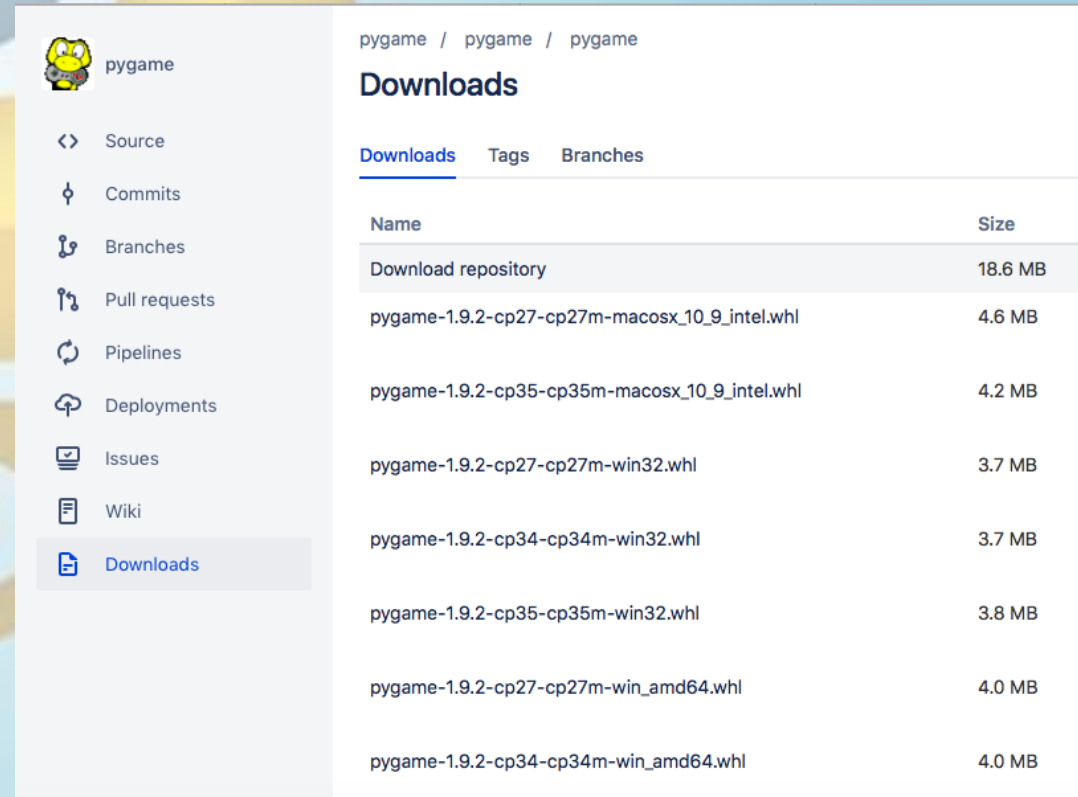
- If error, try pip3 instead of pip
- If neither is installed, go to <https://bootstrap.pypa.io/get-pip.py>
- Save the file as get-pip.py and run
\$ python get-pip.py
- Check pip version again

INSTALLING PYGAME

- Pygame is cross-platform set of Python module designed for writing video games
- @Bitbucket <https://bitbucket.org/pygame/pygame/downloads>
- Download matched version of Python (python3 –version)
- pygame-1.9.2-cp34-cp34m-win_amd64.whl is for Python 3.4 version with 64bit Window
 - copy the .whl to project directory
 - from cmd,

```
python3 -m pip install pygame-1.*.***.whl -- user
```
 - to check

```
>> import pygame
```



The screenshot shows the Bitbucket repository page for Pygame. The left sidebar contains navigation links: Source, Commits, Branches, Pull requests, Pipelines, Deployments, Issues, Wiki, and Downloads (which is highlighted). The main content area is titled 'Downloads' and shows a table of available wheel files for different Python versions and operating systems.

Name	Size
Download repository	18.6 MB
pygame-1.9.2-cp27-cp27m-macosx_10_9_intel.whl	4.6 MB
pygame-1.9.2-cp35-cp35m-macosx_10_9_intel.whl	4.2 MB
pygame-1.9.2-cp27-cp27m-win32.whl	3.7 MB
pygame-1.9.2-cp34-cp34m-win32.whl	3.7 MB
pygame-1.9.2-cp35-cp35m-win32.whl	3.8 MB
pygame-1.9.2-cp27-cp27m-win_amd64.whl	4.0 MB
pygame-1.9.2-cp34-cp34m-win_amd64.whl	4.0 MB

STARTER

1. What do init()?
2. pygame.display.set_mode
3. mainloop() is controlled by while loop
4. need event loop; for loop
5. pygame.event.get()
6. pygame.display.flip()

*alien_
invasion.py*

```
import sys

import pygame

def run_game():
    # Initialize game and create a screen object.
    ❶ pygame.init()
    ❷ screen = pygame.display.set_mode((1200, 800))
    pygame.display.set_caption("Alien Invasion")

    # Start the main loop for the game.
    ❸ while True:

        # Watch for keyboard and mouse events.
        ❹ for event in pygame.event.get():
            ❺ if event.type == pygame.QUIT:
                sys.exit()

        # Make the most recently drawn screen visible.
        ❻ pygame.display.flip()

run_game()
```

SETTING BACKGROUND COLOR

```
import sys
```

```
import pygame
```

```
def run_game():
```

```
    pygame.init()
```

```
    screen = pygame.display.set_mode((1200,800))
```

```
    pygame.display.set_caption("Alien Invasion")
```

```
    #set background color here
```

```
    while True:
```

```
        for event in pygame.event.get():
```

```
            if event.type==pygame.QUIT:
```

```
                sys.exit()
```

```
            pygame.display.flip()
```

```
run_game()
```

- Create color tuple
 - red = (255,0,0)
 - white =(255,255,255)
- Fill command
 - `screen.fill(red)`
- Update display
 - `pygame.display.update()`



CREATE SETTING CLASS

```
alien_
invasion.py  --snip--
import pygame

from settings import Settings

def run_game():
    # Initialize pygame, settings, and screen object.
    pygame.init()
    ❶ ai_settings = Settings()
    ❷ screen = pygame.display.set_mode(
        (ai_settings.screen_width, ai_settings.screen_height))
    pygame.display.set_caption("Alien Invasion")

    # Start the main loop for the game.
    while True:
        --snip--
        # Redraw the screen during each pass through the loop.
        ❸ screen.fill(ai_settings.bg_color)

        # Make the most recently drawn screen visible.
        pygame.display.flip()

run_game()
```

ADDING SHIP OBJECT

```
ship.py  import pygame

        class Ship():

            def __init__(self, screen):
                """Initialize the ship and set its starting position."""
                self.screen = screen

                # Load the ship image and get its rect.
                ❶ self.image = pygame.image.load('images/ship.bmp')
                ❷ self.rect = self.image.get_rect()
                ❸ self.screen_rect = screen.get_rect()

                # Start each new ship at the bottom center of the screen.
                ❹ self.rect.centerx = self.screen_rect.centerx
                self.rect.bottom = self.screen_rect.bottom

                ❺ def blitme(self):
                    """Draw the ship at its current location."""
                    self.screen.blit(self.image, self.rect)
```


EXPLANATION

- Instead of drawing, we use image and load that image on screen

- Pay attention licensing

- .bmp is easiest to configure

1. `pygame.image.load()` `#returns surface for ship`

2. `sef.rect`

3. `self_screen.rect`

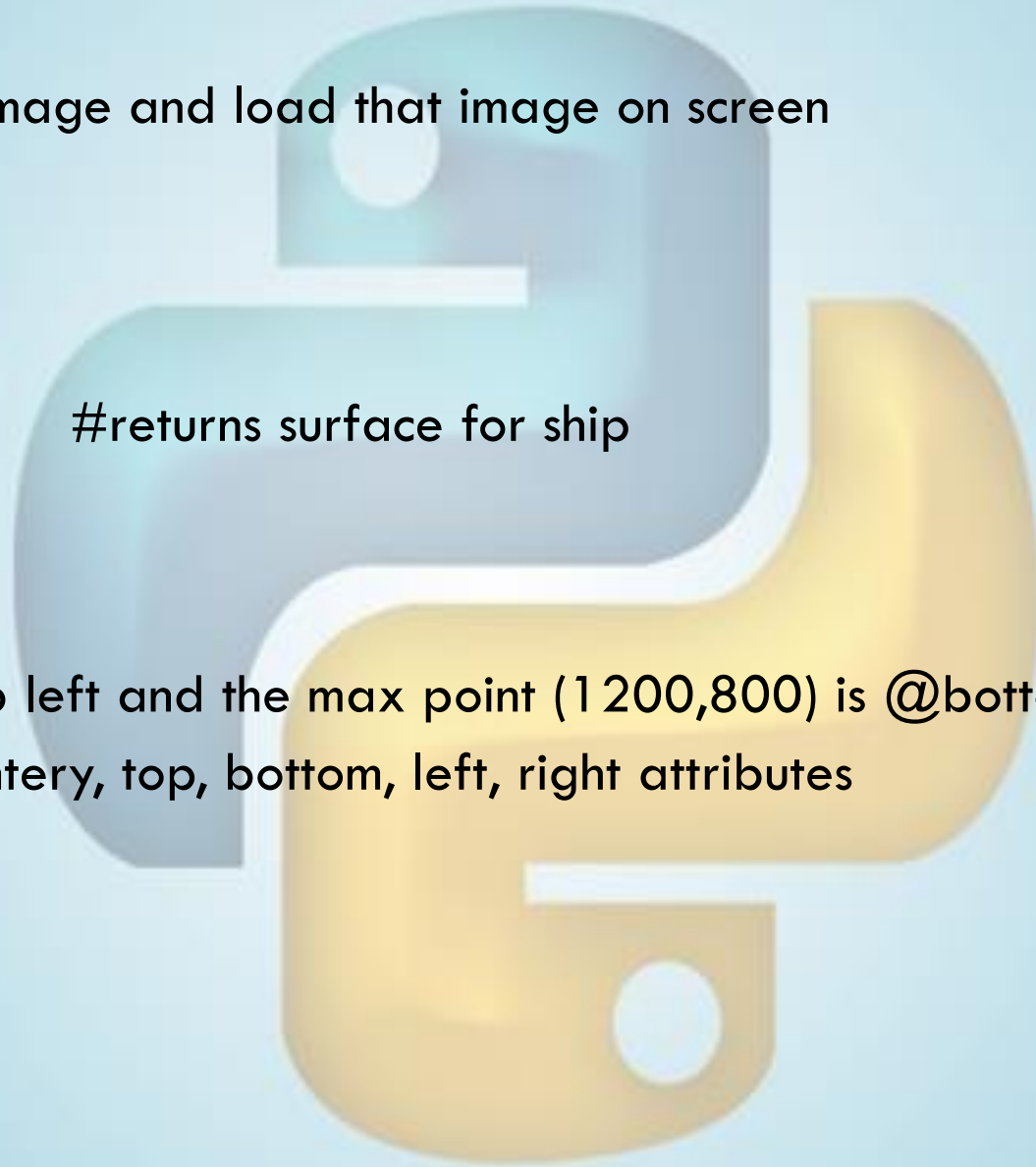
- Python origin (0,0) is @ top left and the max point (1200,800) is @bottom right

- rect has center, centerx, centery, top, bottom, left, right attributes

4. `sel.rect.centerx` & `bottom`

5. `screen.blit`

- bit blit the ship object



UPDATE ALIEN INVASION

*alien_
invasion.py*

```
--snip--
from settings import Settings
from ship import Ship

def run_game():
    --snip--
    pygame.display.set_caption("Alien Invasion")

    # Make a ship.
    ❶ ship = Ship(screen)

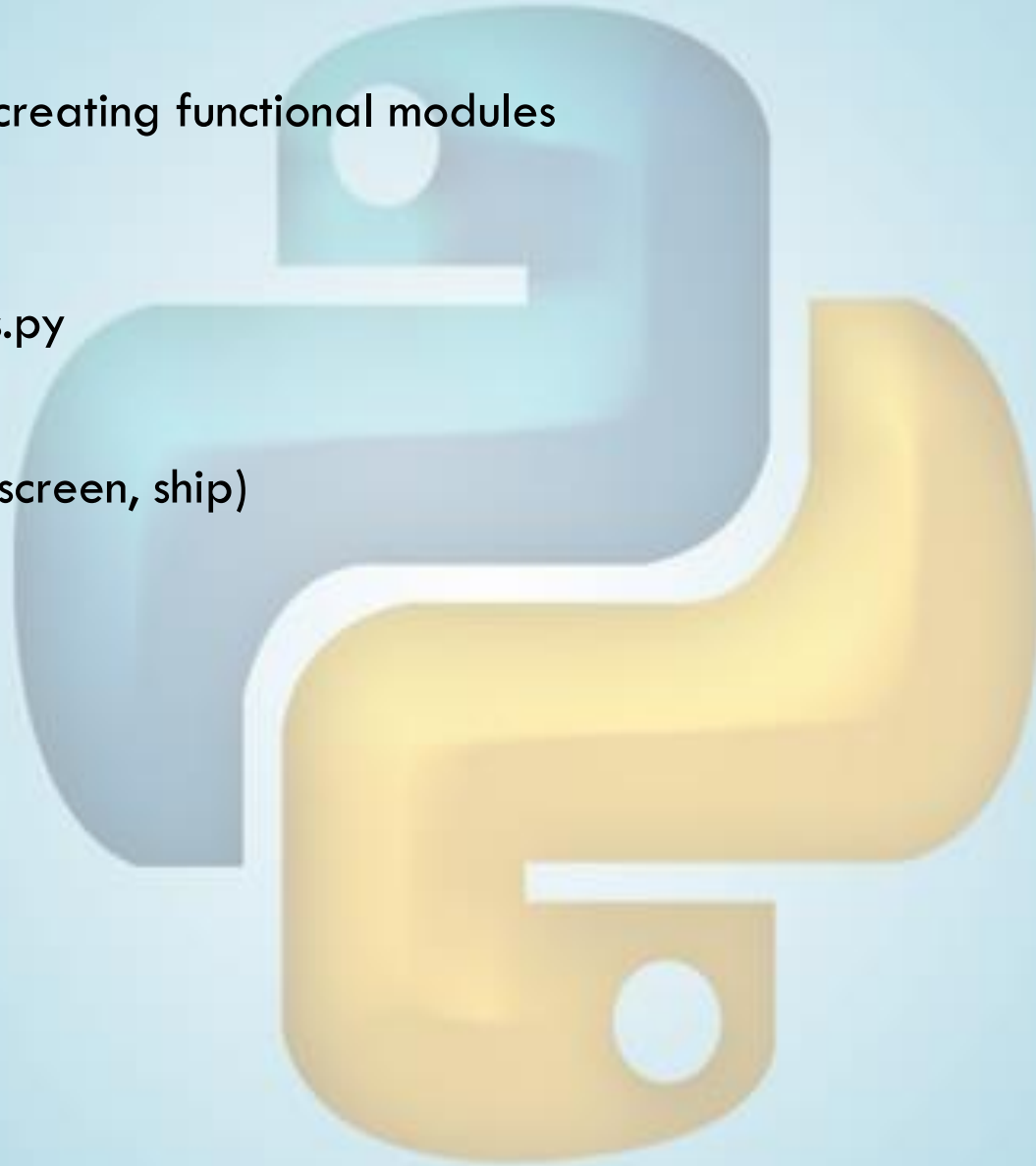
    # Start the main loop for the game.
    while True:
        --snip--
        # Redraw the screen during each pass through the loop.
        screen.fill(ai_settings.bg_color)
        ❷ ship.blitme()

        # Make the most recently drawn screen visible.
        pygame.display.flip()

run_game()
```

REFACTORING THE PROGRAM

- Simplifies the code structure by creating functional modules
- Suitable for larger projects
- Create events in `game_functions.py`
 - `def check_events()`
 - `def update_screen(ai_setting, screen, ship)`



REFACTORING THE PROGRAM

*game_
functions.py*

```
import sys

import pygame

def check_events():
    """Respond to keypresses and mouse events."""
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            sys.exit()

def update_screen(ai_settings, screen, ship):
    """Update images on the screen and flip to the new screen."""
    # Redraw the screen during each pass through the loop.
    screen.fill(ai_settings.bg_color)
    ship.blitme()

    # Make the most recently drawn screen visible.
    pygame.display.flip()
```

ALIEN_INVASION.PY

```
#import game_functions.py module
```

```
import pygame
```

```
def run_game()
```

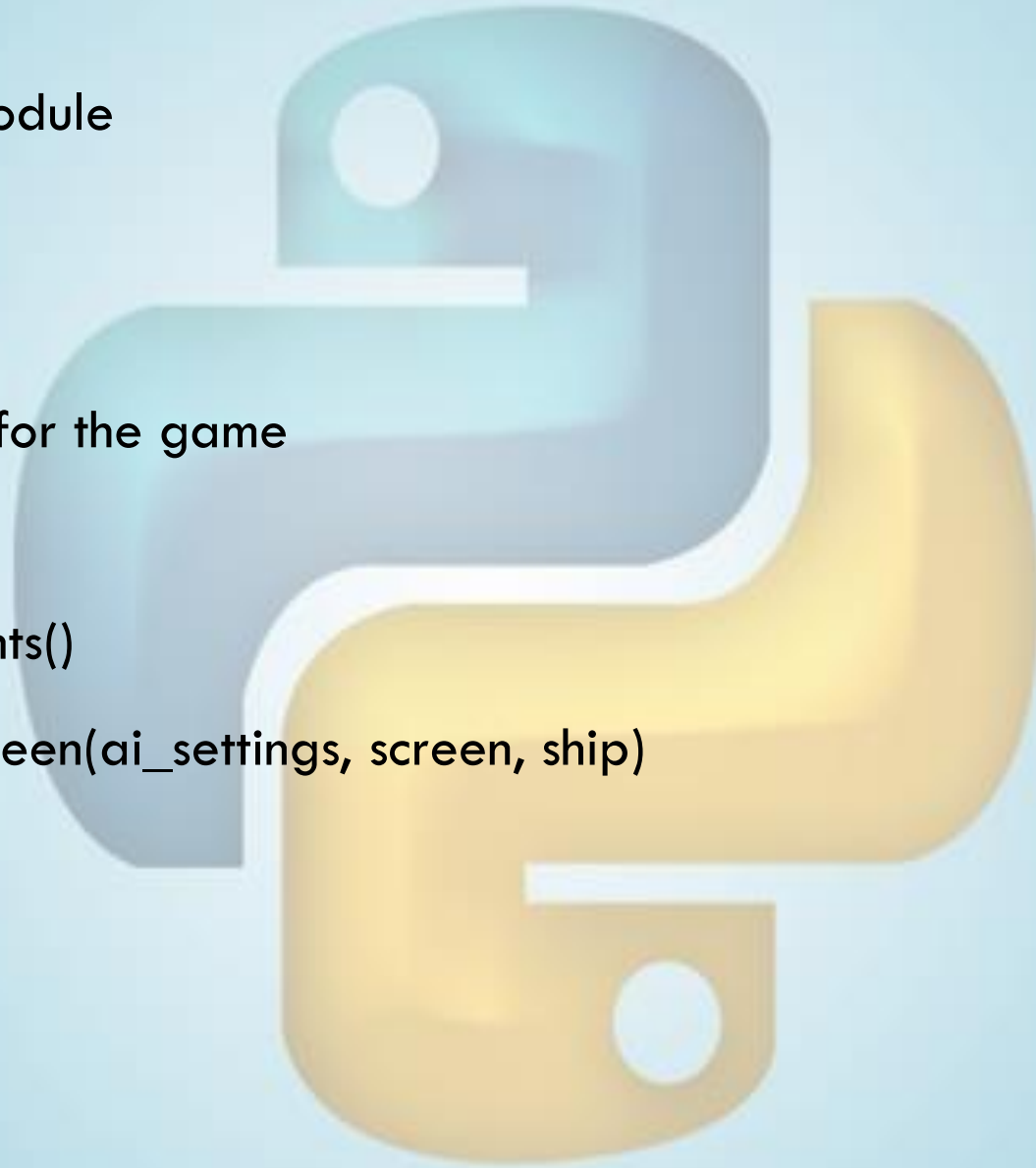
```
    #Start the main loop for the game
```

```
    while True:
```

```
        gf.check_events()
```

```
        gf.update_screen(ai_settings, screen, ship)
```

```
run_game()
```



EXERCISE

TRY IT YOURSELF

12-1. Blue Sky: Make a Pygame window with a blue background.

12-2. Game Character: Find a bitmap image of a game character you like or convert an image to a bitmap. Make a class that draws the character at the center of the screen and match the background color of the image to the background color of the screen, or vice versa.

ENOUGH FOR TODAY

